Section 14: Bit Manipulation and Math

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: LeetCode DSA Foundations for Interviews

Section 14: Bit Manipulation and Math

- SLIDE 129: Section Intro, Bit Manipulation and Math
- SLIDE 130: Essential Bit Operations and XOR
- SLIDE 131: Worked Example: Single Number
- SLIDE 131B: Extension: Two Unique Elements
- SLIDE 132: Power-of-Two and Bit Counting
- SLIDE 133: Math Patterns for Interviews
- SLIDE 134: Wrong vs Right: Bit Manipulation and Math Mistakes
- SLIDE 135: Interview Practice: Bit Manipulation
- SLIDE 136: Course-Wide Pattern Recognition Map
- SLIDE 137: Section Cheat Sheet

C11: LeetCode DSA Foundations for Interviews

Section 14: Bit Manipulation and Math

SLIDE 129: Section Intro, Bit Manipulation and Math

Section 14: Bit Manipulation and Math

Bit manipulation and math problems in interviews share a common trait: if you know the trick, the solution is immediate and elegant. If you do not, you default to a hash map or sorting approach that works but uses more space than necessary. These are not patterns you can derive under pressure. They are patterns you learn once and recognize when they appear.

This section is deliberately placed last because the techniques here are best understood as space-optimized alternatives to approaches from earlier sections. XOR replaces a HashSet for finding unique elements. Bit tricks replace loops for power-of-two checks. Gauss's formula replaces scanning for missing numbers. In each case, the earlier technique works and is acceptable, but the bit or math approach demonstrates deeper knowledge.

What We'll Cover:

1. The six bitwise operators and XOR's three critical properties
2. Single Number and its extension to two unique elements
3. Power-of-two checks and bit counting without loops
4. Math patterns: Gauss's formula, modular arithmetic, GCD, overflow prevention
5. Course-wide pattern recognition map connecting all 14 sections

Where This Fits in the Course:

This is the final section. The techniques here do not introduce new algorithmic paradigms. They provide O(1) space alternatives to hash-based approaches from Section 5 and complement the complexity analysis from Section 2. The course-wide pattern recognition map on Slide 136 ties every section together into a single reference, which is the culmination of the pattern-recognition skill this course has been building from Section 1.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 130: Essential Bit Operations and XOR

The Six Bitwise Operators

Bitwise operators work on the binary representation of integers, operating on each bit position independently.

Operator	Symbol	Rule	Example (5 & 3)	Binary
AND	`&`	1 only if both bits are 1	`5 & 3 = 1`	`101 & 011 = 001`
OR	`|`	1 if either bit is 1	`5 | 3 = 7`	`101 | 011 = 111`
XOR	`^`	1 if bits are different	`5 ^ 3 = 6`	`101 ^ 011 = 110`
NOT	`~`	Flip every bit	`~5 = -6`	`~00000101 = 11111010`
Left Shift	`<<`	Shift bits left, fill with 0	`5 << 1 = 10`	`101 << 1 = 1010`
Right Shift	`>>`	Shift bits right	`5 >> 1 = 2`	`101 >> 1 = 10`

Practical Effects of Shifting:

Left shift by k multiplies by 2^k: `n << 1` is `n * 2`, `n << 3` is `n * 8`.
Right shift by k divides by 2^k (integer division): `n >> 1` is `n / 2`, `n >> 3` is `n / 8`.

These are faster than multiplication and division at the hardware level, though in interviews the primary use is in bit manipulation algorithms, not as arithmetic shortcuts.

XOR: The Three Properties That Matter

XOR is the most important bitwise operator for interview problems. Everything in this section's worked examples depends on three properties:

```
Property 1 - Cancellation:    a ^ a = 0
    Any number XORed with itself produces 0.
    7 ^ 7 = 0       (0111 ^ 0111 = 0000)

Property 2 - Identity:        a ^ 0 = a
```

```
    Any number XORed with 0 is unchanged.
    7 ^ 0 = 7       (0111 ^ 0000 = 0111)

Property 3 - Commutativity:   a ^ b = b ^ a
    Order and grouping do not matter.
    (a ^ b) ^ c = a ^ (b ^ c)
```

The Combined Effect:

When you XOR a collection of numbers, every number that appears an even number of times cancels itself out (Property 1). Only numbers appearing an odd number of times survive (Property 2). The order in which you process them does not matter (Property 3).

```
4 ^ 1 ^ 2 ^ 1 ^ 2
= (1 ^ 1) ^ (2 ^ 2) ^ 4       regroup by commutativity
= 0 ^ 0 ^ 4                    cancellation
= 4                             identity
```

This is the foundation of the Single Number problem and its variants.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 131: Worked Example: Single Number

Problem: Single Number (LeetCode 136)

Given a non-empty array of integers where every element appears exactly twice except for one, find the single element that appears only once.

Approach 1: Hash Set, O(n) Time, O(n) Space

Add elements to a HashSet. If the element is already present, remove it. After processing all elements, the set contains only the unique element.

```java
public int singleNumber(int[] nums) {
    Set<Integer> set = new HashSet<>();
```

```
        for (int num : nums) {
            if (!set.add(num)) {
                set.remove(num);
            }
        }
        return set.iterator().next();
    }
```

This works, but uses O(n) space.

Approach 2: XOR, O(n) Time, O(1) Space

XOR all elements together. Every duplicate pair cancels to 0. The unique element remains.

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num;
    }
    return result;
}
```

Trace through [4, 1, 2, 1, 2]:

```
result = 0
result ^= 4  →  0 ^ 4 = 4   (000 ^ 100 = 100)
result ^= 1  →  4 ^ 1 = 5   (100 ^ 001 = 101)
result ^= 2  →  5 ^ 2 = 7   (101 ^ 010 = 111)
result ^= 1  →  7 ^ 1 = 6   (111 ^ 001 = 110)
result ^= 2  →  6 ^ 2 = 4   (110 ^ 010 = 100)

Result: 4
```

The 1s canceled (appeared at steps 2 and 4). The 2s canceled (appeared at steps 3 and 5). Only 4 survived.

Comparison:

Approach	Time	Space	When to Use
Hash Set	O(n)	O(n)	When you do not know the XOR trick, or when the problem involves non-integer types
Sorting + adjacent check	O(n log n)	O(1)	When you know sorting but not XOR
XOR	O(n)	O(1)	Optimal. Every element appears exactly twice except one.

Both the hash set and XOR approaches are acceptable in interviews. The XOR approach demonstrates deeper knowledge and is the stronger answer when the problem structure matches (all duplicates appear exactly twice).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 131B: Extension: Two Unique Elements

Problem: Single Number III (LeetCode 260)

Given an array where every element appears exactly twice except for two elements that appear once each, find both unique elements.

XORing everything gives `a ^ b` (where a and b are the two unique elements), not the individual values. We need one more step.

The Technique: Split by Differing Bit

Since a and b are different, `a ^ b` has at least one bit set to 1. That bit is a position where a and b differ. We can use it to split all elements into two groups: elements with that bit set, and elements without. Each group contains exactly one unique element and some pairs (which cancel within their group).

Full Java Solution:

```
public int[] singleNumber(int[] nums) {
    int xorAll = 0;
    for (int num : nums) {
```

```
            xorAll ^= num;
        }

        int diffBit = xorAll & (-xorAll);

        int a = 0, b = 0;
        for (int num : nums) {
            if ((num & diffBit) != 0) {
                a ^= num;
            } else {
                b ^= num;
            }
        }
        return new int[]{a, b};
    }
```

How `xorAll & (-xorAll)` Works:

- `xorAll` is the two's complement (flip all bits, add 1). ANDing a number with its negation isolates the lowest set bit.

```
If xorAll = 6 (binary 110):
  -xorAll = ...11111010 (two's complement)
  xorAll & (-xorAll) = 010 = 2

Bit position 1 (value 2) is where a and b differ.
```

Trace through [1, 2, 1, 3, 2, 5]:

```
Step 1: XOR all → 1^2^1^3^2^5 = 3^5 = 6 (binary 110)
Step 2: Isolate lowest set bit → 6 & (-6) = 2 (binary 010)

Step 3: Split by bit 1 (value 2):
  Bit set (& 2 != 0):     2, 3, 2      → XOR: 2^3^2 = 3
  Bit not set (& 2 == 0): 1, 1, 5      → XOR: 1^1^5 = 5

Result: [3, 5]
```

The pairs (1,1) and (2,2) cancel within their respective groups. Each group is left with exactly one unique element.

Time O(n), space O(1). Two passes through the array: one to compute the combined XOR, one to split and compute individual values.

Interview Tip: This problem is rarely asked at the fresher level, but understanding the technique demonstrates strong bit manipulation fluency. If asked Single Number and you solve it quickly, the interviewer may follow up with this variant. Knowing it exists and being able to sketch the approach is a positive signal even if you do not implement it perfectly.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 132: Power-of-Two and Bit Counting

Check if n is a Power of Two

A power of two in binary has exactly one bit set: 1, 2, 4, 8, 16 are 1, 10, 100, 1000, 10000 in binary.

The trick: `n & (n - 1)` clears the lowest set bit. If n is a power of two, it has only one set bit, so clearing it gives 0.

```
public boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Why `n > 0` is Required:

`n = 0` would satisfy `(n & (n-1)) == 0` but zero is not a power of two. Negative numbers in two's complement have multiple set bits.

How `n & (n - 1)` Works:

```
n = 16:  10000
n-1 = 15: 01111
```

```
n & (n-1): 00000  → 0, so 16 is a power of two

n = 12:  01100
n-1 = 11: 01011
n & (n-1): 01000  → 8, not 0, so 12 is not a power of two
```

Subtracting 1 flips all bits from the lowest set bit downward. ANDing with the original clears that lowest set bit and everything below it.

Count Set Bits: Brian Kernighan's Algorithm

Each iteration of `n = n & (n - 1)` clears the lowest set bit. Count iterations until n becomes 0.

```java
public int countSetBits(int n) {
    int count = 0;
    while (n != 0) {
        n = n & (n - 1);
        count++;
    }
    return count;
}
```

For n = 13 (binary 1101): three iterations clear bits at positions 0, 2, and 3. Count = 3. This runs in O(k) where k is the number of set bits, faster than checking all 32 bit positions when the number has few set bits.

Common Bit Tricks Quick Reference:

Operation	Expression	Example
Check if bit k is set	`(n >> k) & 1`	bit 2 of 5 (101): `(5 >> 2) & 1 = 1`
Set bit k	`n | (1 << k)`	set bit 1 of 5 (101): `5 | 2 = 7 (111)`
Clear bit k	`n & ~(1 << k)`	clear bit 2 of 5 (101): `5 & ~4 = 1 (001)`
Toggle bit k	`n ^ (1 << k)`	toggle bit 0 of 5 (101): `5 ^ 1 = 4 (100)`
Get lowest set bit	`n & (-n)`	lowest of 12 (1100): `12 & -12 = 4 (100)`
Clear lowest set bit	`n & (n - 1)`	clear lowest of 12 (1100): `12 & 11 = 8 (1000)`

Operation	Expression	Example
Check power of two	`n > 0 && (n & (n-1)) == 0`	16 (10000): true

You do not need to memorize every trick. The three most commonly tested are: XOR for duplicate cancellation (Slide 131), `n & (n-1)` for power-of-two check and bit counting, and `n & (-n)` for isolating the lowest set bit (Slide 131B).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 133: Math Patterns for Interviews

Gauss's Formula: Sum of 1 to n

```
sum = n * (n + 1) / 2
```

Primary interview application: finding a missing number. If an array contains n distinct numbers from 0 to n with one missing, the missing number is the expected sum minus the actual sum.

```java
public int missingNumber(int[] nums) {
    int n = nums.length;
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int num : nums) {
        actualSum += num;
    }
    return expectedSum - actualSum;
}
```

Time O(n), space O(1). Alternative: XOR approach (XOR all indices 0..n with all array values; pairs cancel, missing number remains). Both are O(n) time O(1)

space. The Gauss approach is easier to explain; the XOR approach avoids potential overflow for very large n.

Modular Arithmetic

When intermediate results could overflow, use modular arithmetic to keep values bounded.

```
(a + b) % m = ((a % m) + (b % m)) % m
(a * b) % m = ((a % m) * (b % m)) % m
```

Common in problems that say “return the answer modulo 10^9 + 7.” Apply the modulus after every addition or multiplication, not just at the end.

```
int MOD = 1_000_000_007;
long result = 0;
for (int i = 0; i < n; i++) {
    result = (result + arr[i]) % MOD;
}
```

Use `long` for intermediate results even when the final answer fits in `int`. Two values up to 10^9 multiplied together can reach 10^18, which overflows `int` (max ~2.1 × 10^9) and even requires care with `long` (max ~9.2 × 10^18).

GCD: Euclidean Algorithm

Greatest common divisor. Used in fraction simplification, LCM computation, and problems involving divisibility.

```
public int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

LCM from GCD: `lcm(a, b) = a / gcd(a, b) * b`. Divide first to avoid overflow.

Integer Overflow Awareness

Integer overflow is a correctness bug, not a performance issue. It produces silently wrong results.

Danger Zone	Wrong Form	Safe Form
Binary search midpoint	`(left + right) / 2`	`left + (right - left) / 2`
Comparator subtraction	`(a, b) -> a - b`	`(a, b) -> Integer.compare(a, b)`
Large running sum	`int sum += val`	`long sum += val`
Product of two ints	`int product = a * b`	`long product = (long) a * b`

The binary search midpoint overflow was covered in Section 8 (Slide 73). The comparator overflow was covered in Section 9 (Slide 83). These are listed here as a consolidated reference because overflow is a math-level concern that spans multiple sections.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 134: Wrong vs Right: Bit Manipulation and Math Mistakes

Mistake 1: Using Hash Map for Single Number When XOR Gives O(1) Space

Wrong: “I will use a HashMap to count frequencies and return the element with count 1.” This is O(n) time O(n) space.

Right: “Every element appears exactly twice except one. XORing all elements cancels the pairs and leaves the unique element. O(n) time, O(1) space.”

Both approaches are correct. The hash map is acceptable. But when the problem structure is “duplicates cancel, one survives,” recognizing XOR demonstrates stronger pattern recognition and produces the optimal space solution.

Mistake 2: Checking Power of Two with a Loop

Wrong:

```java
public boolean isPowerOfTwo(int n) {
    if (n <= 0) return false;
    while (n > 1) {
        if (n % 2 != 0) return false;
        n /= 2;
    }
    return true;
}
```

This is O(log n). It works, but the bit trick is O(1):

```java
return n > 0 && (n & (n - 1)) == 0;
```

The loop approach is not wrong, and an interviewer will accept it. But the bit trick is a stronger answer when the problem is explicitly about binary properties.

Mistake 3: Integer Overflow in Binary Search Midpoint

Wrong:

```java
int mid = (left + right) / 2;
```

If left = 1,500,000,000 and right = 2,000,000,000, their sum is 3,500,000,000, which overflows Java's `int` (max 2,147,483,647). The result wraps to a negative number, producing an invalid array index.

Right:

```java
int mid = left + (right - left) / 2;
```

This computes the difference first (which is always non-negative and smaller than either operand), divides it, then adds to left. No overflow possible.

Mistake 4: Comparator Subtraction Overflow

Wrong:

```java
Arrays.sort(arr, (a, b) -> a - b);
```

If a = Integer.MAX_VALUE and b = -1, then a - b overflows. The sort receives a negative result instead of positive, producing incorrect ordering.

Right:

```
Arrays.sort(arr, (a, b) -> Integer.compare(a, b));
```

`Integer.compare` handles the full range without overflow.

Mistake 5: Using Floating-Point Division When Integer Math Suffices

Wrong:

```
double avg = (double)(a + b) / 2;
if (avg == target) { ... }
```

Floating-point comparison with `==` is unreliable due to precision errors.

Right: If you need to check whether `(a + b) / 2 == target`, use `a + b == 2 * target` to stay in integer arithmetic. Avoid floating-point whenever the problem deals exclusively with integers.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 135: Interview Practice: Bit Manipulation

Practice Prompt

“Every element in this array appears twice except one. Find the single element.” (LeetCode 136)

Expected Walkthrough:

Step 1 (Understand): “The array has n elements. Every value appears exactly twice except one value that appears once. I need to find that unique value.”

Step 2 (Approaches): “I can solve this with a HashMap for frequency counting in O(n) time and O(n) space. But since every duplicate appears exactly twice, I can

use XOR. XORing all elements cancels every pair to 0, leaving only the unique element. O(n) time, O(1) space."

Step 3 (Implement):

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int num : nums) {
        result ^= num;
    }
    return result;
}
```

Step 4 (Verify): "For [2, 3, 2]: result starts at 0, XOR 2 gives 2, XOR 3 gives 1, XOR 2 gives 3. The unique element is 3."

Step 5 (Complexity): "Time O(n), one pass through the array. Space O(1), single integer variable."

Common Interviewer Follow-Ups:

Follow-Up	Expected Response
"Why does XOR work here?"	"XOR has the cancellation property: a ^ a = 0. Every duplicate pair cancels. The identity property a ^ 0 = a means the unique element survives."
"What if elements appeared three times except one?"	"XOR only cancels pairs. For triples, I would use a bit-counting approach: count each bit position modulo 3. The remaining bits form the unique element. Or use a HashMap."
"What if there are two unique elements?"	"XOR all to get a ^ b. Isolate a differing bit with n & (-n). Split elements into two groups by that bit. XOR each group separately to recover a and b."
"Can you do this without XOR?"	"HashSet: add if absent, remove if present. The remaining element is unique. O(n) time, O(n) space. Or sort and check adjacent pairs: O(n log n) time, O(1) space."

Common Fresher Mistakes:

- Not knowing XOR properties and defaulting to HashMap. The HashMap solution is correct and acceptable, but when asked “can you do this in O(1) space?”, you need the XOR approach.
- Over-complicating with sorting or frequency maps when the XOR solution is three lines. Recognizing when a problem has a clean bit manipulation solution is the skill this section builds.
- Trying to apply XOR to problems where it does not fit. XOR cancels pairs. If duplicates appear three times, or if the “duplicate” condition is more complex, XOR alone is not sufficient. Know the boundary of where the trick applies.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 136: Course-Wide Pattern Recognition Map

This is the single most important reference in the course. It maps problem signals to patterns across all 14 sections. When you encounter a new problem, scan the left column for matching signals. The corresponding pattern is your starting point.

Problem Signal to Pattern Map:

Problem Signal	Pattern	Section	Time	Space
“Find pair summing to target” (unsorted)	Hash map complement lookup	Section 5	O(n)	O(n)
“Find pair summing to target” (sorted)	Opposite-direction two pointers	Section 6	O(n)	O(1)
“Three numbers summing to target”	Sort + fix one + two pointers	Section 6	O(n^2)	O(1)
“Contiguous subarray of size K”	Fixed-size sliding window	Section 7	O(n)	O(1)

Problem Signal	Pattern	Section	Time	Space
“Longest/shortest subarray with condition”	Variable-size sliding window	Section 7	O(n)	O(1)
“Count subarrays with exactly K”	atMost(K) - atMost(K-1)	Section 7	O(n)	O(1)
“Subarray sum equals K”	Prefix sum + hash map	Sections 3, 5	O(n)	O(n)
“Maximum contiguous subarray sum”	Kadane’s algorithm	Section 3	O(n)	O(1)
“Search in sorted array”	Binary search (Template 1)	Section 8	O(log n)	O(1)
“First/last position in sorted array”	Binary search (Template 2, boundary)	Section 8	O(log n)	O(1)
“Minimum X such that condition”	Binary search on answer space	Section 8	O(n log R)	O(1)
“Search in rotated sorted array”	Modified binary search	Section 8	O(log n)	O(1)
“In place, remove, deduplicate”	Same-direction two pointers	Section 6	O(n)	O(1)
“Count occurrences, frequency”	Hash map frequency counting	Section 5	O(n)	O(n)
“Group by property”	Hash map property-to-list	Section 5	O(n)	O(n)
“Check existence, deduplicate”	HashSet	Section 5	O(n)	O(n)
“Anagram, character frequency”	int[26] or HashMap frequency	Section 4	O(n)	O(1) or O(k)
“Palindrome check”	Two pointers from both ends	Sections 4, 6	O(n)	O(1)

Problem Signal	Pattern	Section	Time	Space
“Valid brackets, balanced nesting”	Stack: push expected, pop and compare	Section 10	O(n)	O(n)
“Next greater/smaller element”	Monotonic stack	Section 10	O(n)	O(n)
“Sliding window maximum”	Deque (decreasing value order)	Section 10	O(n)	O(k)
“Merge overlapping intervals”	Sort by start + linear merge	Section 9	O(n log n)	O(n)
“Reverse linked list”	Three pointers: prev, curr, next	Section 11	O(n)	O(1)
“Cycle in linked list”	Floyd’s fast-slow pointers	Section 11	O(n)	O(1)
“Middle of linked list”	Fast-slow pointers	Section 11	O(n)	O(1)
“Merge sorted lists”	Dummy node + comparison	Section 11	O(n+m)	O(1)
“Generate all permutations”	Backtracking with used array	Section 12	O(n!)	O(n)
“Generate all subsets”	Backtracking with start index	Section 12	O(2^n)	O(n)
“Generate combinations of size k”	Backtracking with start + size	Section 12	O(C(n,k))	O(k)
“Find all valid configurations”	Backtracking with pruning	Section 12	Varies	Varies
“Locally optimal, never revisit”	Greedy	Section 13	O(n) or O(n log n)	O(1)
“Maximum non-overlapping intervals”	Greedy: sort by end time	Section 13	O(n log n)	O(1)

Problem Signal	Pattern	Section	Time	Space
“Can you reach the end?”	Greedy: track farthest reachable	Section 13	O(n)	O(1)
“Unique element among duplicates”	XOR all elements	Section 14	O(n)	O(1)
“Missing number from 0..n”	Gauss’s formula or XOR	Section 14	O(n)	O(1)
“Is power of two?”	`n & (n-1) == 0`	Section 14	O(1)	O(1)

How to Use This Map:

1. Read the problem. Identify the constraint on n (determines acceptable complexity).
2. Scan the signals column for phrases that match the problem statement.
3. The pattern column tells you the technique. The section column tells you where to review it.
4. If multiple signals match, start with the most specific one. “Subarray sum equals K” is more specific than “contiguous subarray with condition.”
5. If the problem does not match any signal cleanly, start with brute force and look for the optimization: is there a nested loop that can be replaced with a hash map? Is the input sorted? Can you maintain a running state instead of recomputing?

This map is a starting point, not a decision oracle. Some problems combine multiple patterns (prefix sum + hash map, sort + two pointers, sliding window + deque). The pattern tells you where to start; your problem-solving framework from Section 1 guides the rest.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 137: Section Cheat Sheet

Bit Operation Quick Reference

Operation	Expression	Cost
XOR all elements	Loop with `result ^= num`	O(n)
Check power of two	`n > 0 && (n & (n-1)) == 0`	O(1)
Count set bits	Loop with `n = n & (n-1)`, count	O(k), k = set bits
Isolate lowest set bit	`n & (-n)`	O(1)
Clear lowest set bit	`n & (n - 1)`	O(1)
Check bit k	`(n >> k) & 1`	O(1)

XOR Pattern: Duplicates Cancel, Unique Survives

Problem	Technique	Time	Space
One unique, rest appear twice	XOR all	O(n)	O(1)
Two unique, rest appear twice	XOR all, isolate differing bit, split and XOR groups	O(n)	O(1)
One unique, rest appear three times	Bit count modulo 3 per position (or HashMap)	O(n)	O(1) or O(n)

Math Formulas

Formula	Expression	Interview Use
Sum of 1 to n	`n * (n + 1) / 2`	Missing number: expected minus actual
Modular addition	`(a + b) % m = ((a % m) + (b % m)) % m`	“Return answer modulo 10^9 + 7”
GCD	`gcd(a, b) = gcd(b, a % b)`, base: `gcd(a, 0) = a`	Fraction simplification, LCM
LCM from GCD	`a / gcd(a, b) * b`	Divide first to prevent overflow

Integer Overflow Prevention

Danger	Wrong	Safe
Binary search mid	`(left + right) / 2`	`left + (right - left) / 2`

Danger	Wrong	Safe
Comparator	`(a, b) -> a - b`	`(a, b) -> Integer.compare(a, b)`
Large sums	`int sum`	`long sum`
Products	`int p = a * b`	`long p = (long) a * b`

How This Section Connects to Everything Else

- Section 2 (Complexity): bit operations are O(1). Recognizing when a loop can be replaced by a bit trick is a complexity optimization.
- Section 3 (Arrays): missing number can be solved with Gauss's formula instead of sorting or hashing.
- Section 5 (Hash Maps): XOR is the O(1) space alternative to HashSet for duplicate-cancellation problems. Hash maps remain the general-purpose tool when duplicates appear more than twice or when associated data is needed.
- Section 8 (Binary Search): overflow prevention for the midpoint calculation is a math concern that applies every time you write binary search.
- Section 9 (Sorting): comparator overflow from `a - b` is a math concern that applies every time you write a custom sort.

Common Interview Mistakes

Mistake	Why It's Wrong	Better Approach
HashMap for Single Number	Works but O(n) space when O(1) is possible	XOR cancels duplicate pairs, O(1) space
Loop to check power of two	O(log n) when O(1) exists	`n > 0 && (n & (n-1)) == 0`
`(left + right) / 2`	Overflows when left + right > Integer.MAX_VALUE	`left + (right - left) / 2`
`(a, b) -> a - b` comparator	Overflows for extreme values	`Integer.compare(a, b)`
Floating-point == comparison	Precision errors give wrong results	Stay in integer arithmetic when possible
Applying XOR to non-pair duplicates	XOR only cancels even-count occurrences	Use bit counting mod k, or HashMap

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina